Linux 程序设计

实验二　Shell 程序设计实验

附：Redis 服务器搭建

学　　院	计算机学院
专　　业	计算机科学与技术
学　　号	B24040514
姓　　名	张瀚文
课程名称	Linux 程序设计
实验日期	2026 年 11 月 11 日

实验目的

1. 掌握 Shell 程序中变量的定义与使用，分清**局部变量**与**环境变量**的区别，理解环境变量可被子进程继承的机制。
2. 掌握 Shell 中**单引号、双引号、反引号**的作用差异，理解引号对变量替换、命令替换和通配符展开的屏蔽效果。
3. 能熟练使用 `if`、`case`、`for`、`while` 等控制语句编写 Shell 脚本，并用 `test` / `[ ]` 完成条件判断。
4. 掌握 Shell **函数**的定义、调用与返回值（退出状态码 `$?`）的使用。
5. 独立完成 Redis 服务端的安装、配置、启动与远程连接验证，为后续在 Linux 下做服务部署打基础。

实验环境

项目	内容
宿主机	Windows，用于编辑文档、截图与提交报告
虚拟机	Ubuntu 22.04 LTS，主机名 `azrael-virtual-machine`，用户名 `azrael`
Shell	bash 5.1
虚拟机 IP	`192.168.114.128`（NAT 方式）
服务器软件	`redis-server`（apt 安装，由 systemd 托管，监听 `0.0.0.0:6379`）
客户端工具	终端；RedisInsight（图形化客户端，运行在宿主机上）

实验一　Redis 服务器搭建

3.1 安装 redis-server

Ubuntu 官方源中已经包含 redis，直接使用 apt 安装，由 systemd 接管服务：

```
sudo apt update
sudo apt install -y redis-server
```

安装过程中会安装 `redis-server`、`redis-tools` 两个包，并自动创建 `redis` 用户、注册 `redis-server.service` 单元。安装日志末尾可以看到 systemd 正在处理 `redis-server.service` 的触发器。

安装完成后确认版本与可执行文件位置：

```
redis-server --version
which redis-server redis-cli
```

3.2 修改配置 `/etc/redis/redis.conf`

默认配置只允许本机（`127.0.0.1`）访问，而本实验需要用宿主机上的 RedisInsight 通过虚拟机的局域网地址连接，因此必须改监听地址。修改前先备份：

```
sudo cp /etc/redis/redis.conf /etc/redis/redis.conf.bak
sudo vim /etc/redis/redis.conf
```

需要修改 / 确认的四处：

配置项	默认值	修改为	作用
`bind`	`127.0.0.1 -::1`	`0.0.0.0`	监听所有网卡，允许宿主机访问
`protected-mode`	`yes`	`no`	关闭保护模式，否则跨机访问会被拒绝
`port`	`6379`	`6379`	保持默认端口
`requirepass`	注释状态	按需设置，如 `requirepass 123456`	设置访问密码，客户端需 AUTH

> 若设置了 `requirepass`，RedisInsight 中建立连接时必须填写密码；若未设置密码，则关闭 `protected-mode` 后即可直连。两者不能同时「有密码 + 保护模式开启」还指望远程可写，实验时按需取一种。

`bind` 与 `protected-mode` 的修改效果可以在 `systemctl status` 的进程行里直接验证——进程参数显示为 `redis-server 0.0.0.0:6379`，说明配置已生效。

3.3 重启服务并验证运行状态

```
sudo systemctl restart redis-server
sudo systemctl status redis-server
# 可选：设置开机自启
sudo systemctl enable redis-server
```

`systemctl status` 的输出是本次验证的核心证据：

```
● redis-server.service - Advanced key-value store
     Loaded: loaded (/lib/systemd/system/redis-server.service; enabled; vendor preset: enabled)
     Active: active (running) since Wed 2026-10-21 16:18:02 CST; 10s ago
       Docs: http://redis.io/documentation,
             man:redis-server(1)
   Main PID: 3822 (redis-server)
     Status: "Ready to accept connections"
      Tasks: 5 (limit: 4540)
     Memory: 2.8M
        CPU: 107ms
     CGroup: /system.slice/redis-server.service
             └─3822 "/usr/bin/redis-server 0.0.0.0:6379" ...
10月 21 16:18:02 azrael-virtual-machine systemd[1]: Starting Advanced key-value store...
10月 21 16:18:02 azrael-virtual-machine systemd[1]: Started Advanced key-value store.
```

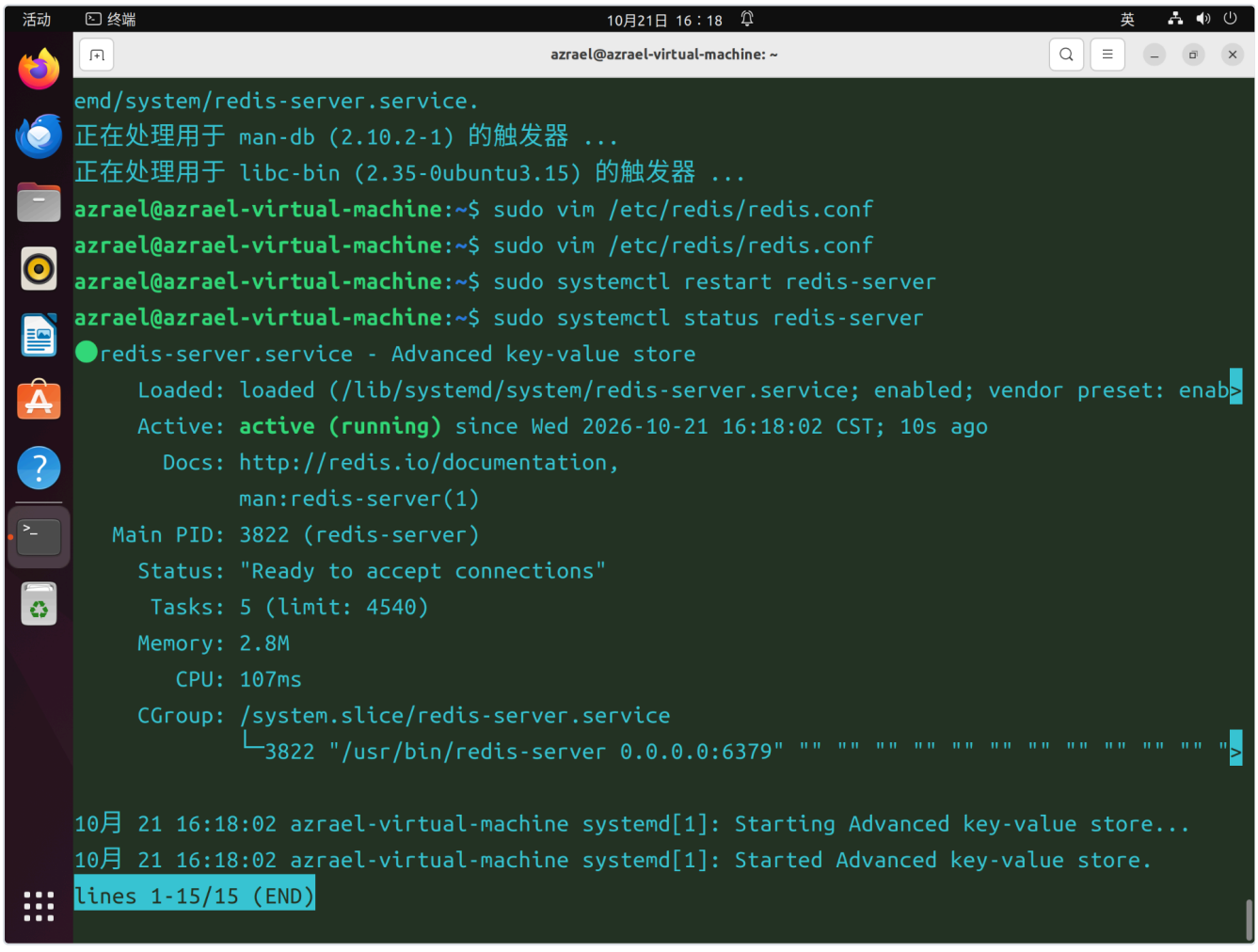


图 3-1 `systemctl status redis-server`：Active 为 active (running)，Status 为 “Ready to accept connections”，进程监听 0.0.0.0:6379

补充验证命令：

```
# 端口是否在监听（应该看到 0.0.0.0:6379）
ss -lntp | grep 6379

# 本机连通性测试
redis-cli ping              # 期望 PONG
redis-cli -h 192.168.114.128 ping

# 设置密码后需要先认证
redis-cli -a 123456 ping    # 期望 PONG
```

3.4 用 RedisInsight 远程连接验证

宿主机上打开 RedisInsight，选择 *Connect existing database*，Connection Type 选 *Standalone*，填写：

- Host：`192.168.114.128`
- Port：`6379`
- Database Alias：`linux-lesson`
- Password：与 `requirepass` 一致（未设密码则留空）

保存后数据库列表中新增一条 `linux-lesson 192.168.114.128:6379 Standalone`，右下角弹出提示 *Database has been added / linux-lesson has been added to Redis Insight*，*Last connection* 显示 “less than a minute ago”，说明远程连接成功。

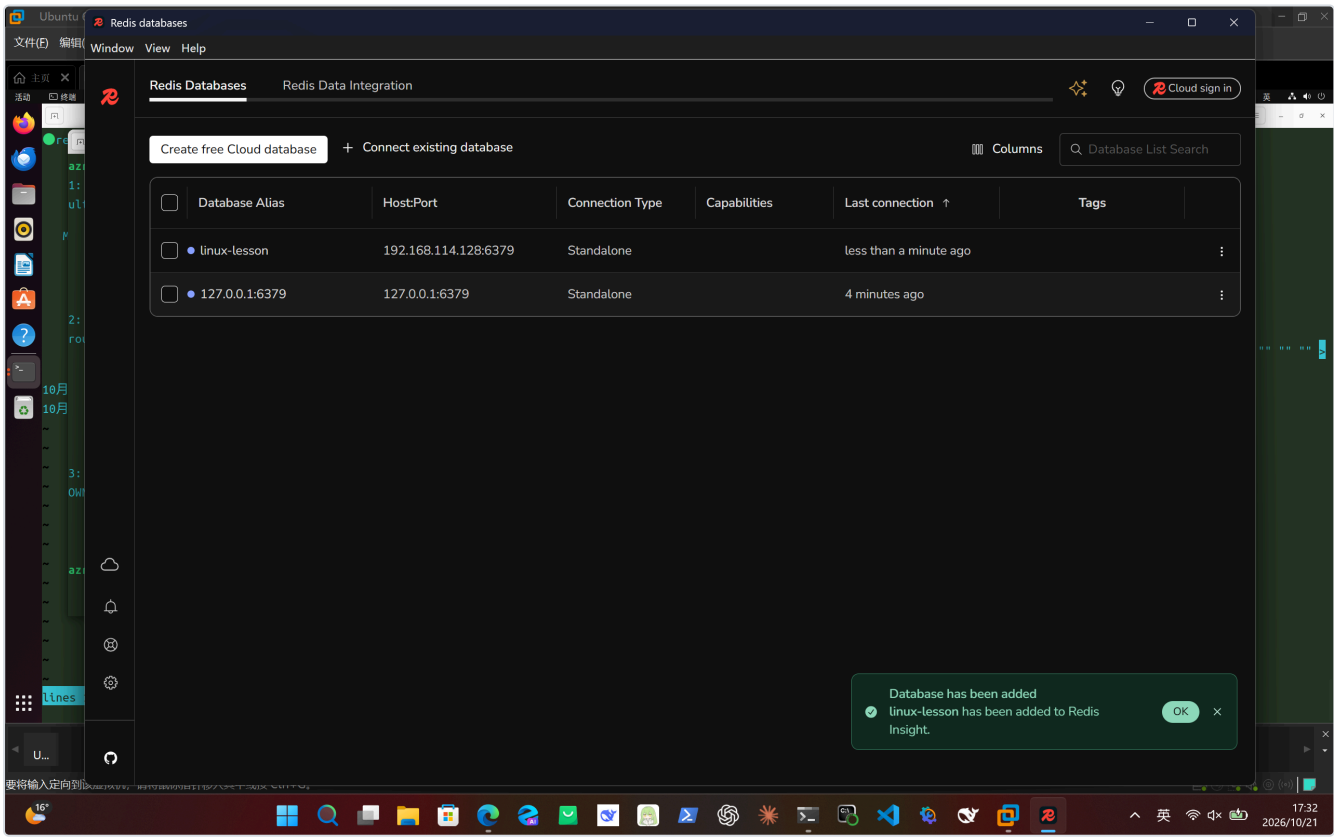


图 3-2　RedisInsight 数据库列表：`linux-lesson` → 192.168.114.128:6379（Standalone），与 `127.0.0.1:6379` 并列

3.5 本节小结与遇到的问题

问题	原因	解决办法
虚拟机内 `redis-cli ping` 正常，宿主机连不上	`bind 127.0.0.1` 只监听回环地址	改为 `bind 0.0.0.0` 并重启服务
连接被拒绝 / 提示保护模式	`protected-mode yes`	改为 `no`（或保留保护模式但设置 `requirepass`）
改了配置但进程仍是 `127.0.0.1:6379`	只改文件没重启服务	`sudo systemctl restart redis-server`
宿主机 ping 不通虚拟机	网卡模式是「仅主机」但网段不一致	核对 NAT 网段，虚拟机 IP 为 192.168.114.128

小结：Redis 在 Ubuntu 下以 systemd 服务的形式运行，配置集中在 `/etc/redis/redis.conf`。让外部客户端可访问需要同时满足「`bind` 包含外网地址」「`protected-mode no` 或设置密码」「防火墙放行 6379」三个条件，缺一个都会连接失败。

实验二　Shell 程序设计

4.1 局部变量与环境变量

4.1.1 实验过程

在当前 shell 中定义两个变量，只把 `var2` 导出为环境变量：

```
var1="welcome to use local variable"
var2="I am global varible"
export var2
```

`export` 的作用是给变量打上「环境变量」标记，使它被**写进环境表**，从而能被当前 shell 派生的子进程继承。可以用下面的命令确认这个标记：

```
export -p | grep var2      # declare -x var2="I am global varible"
env | grep var2            # 环境变量表中出现 var2，而 var1 不在
```

随后创建子 bash（子进程）验证继承关系：

```
bash
echo "$var1"        # 空行：子 shell 的环境里没有 var1
echo "$var2"        # I am global varible
exit
```

退出子 shell 回到父 shell 后，两个变量依然可以使用：

```
echo "$var1"        # welcome to use local variable
echo "$var2"        # I am global varible
```

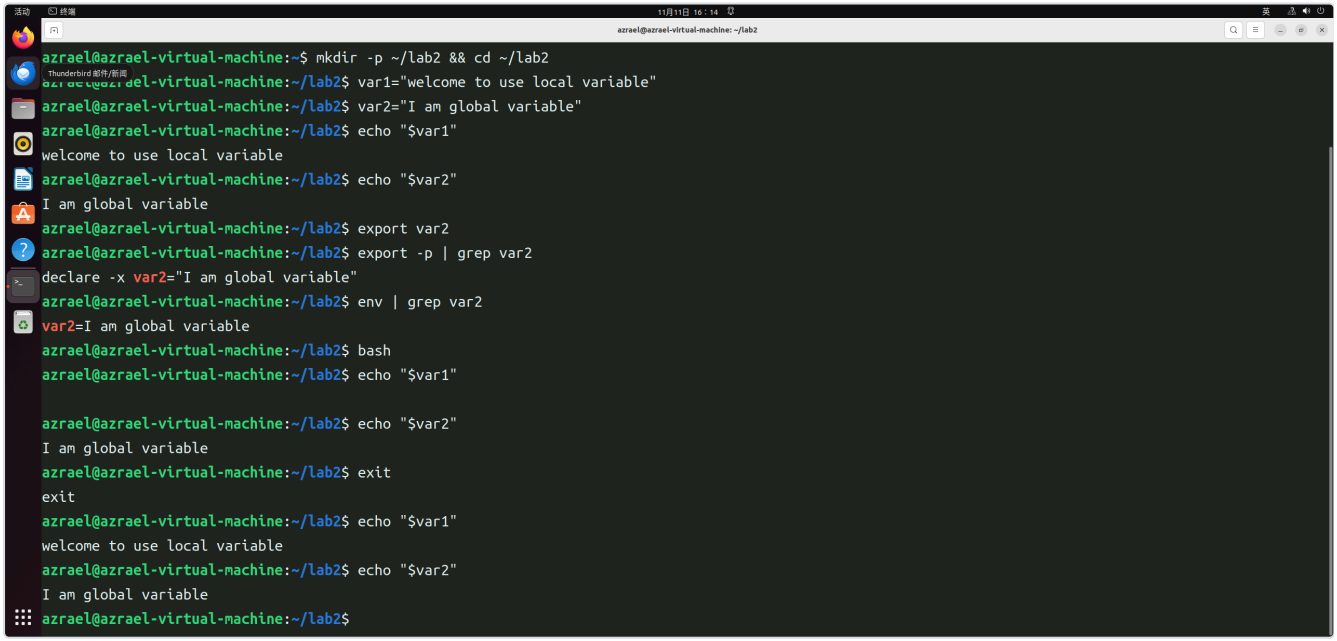


图 4-1　`export -p` 与 `env` 中能看到 var2；子 shell 中 `$var1` 为空、`$var2` 有值；`exit` 回父 shell 后两者都有值

4.1.2 结论

- **普通变量（局部变量）**：只存在于当前 shell 进程的变量表里，子进程不可见。
- **环境变量**：用 `export` 导出后进入环境表，会被 `fork` + `exec` 出的子进程继承，因此 `bash`、`ls` 等子进程都能读到 `$var2`。
- 环境变量「继承」是**单向**的：父进程导出后子进程能看到；反过来，子在进程中定义/修改的变量不会影响父进程（除非再次 `source`）。
- 常用的持久化方式是在 `~/.bashrc` 或 `/etc/profile` 中写 `export 变量名=值`，登录时由 shell 自动导出。

4.2 引号的作用

4.2.1 实验过程与观察

`~/lab2` 目录下已有若干 `.sh` 文件，用它来观察通配符展开：

```
cd ~/lab2

echo *            # 通配符先被 shell 展开 → 列出所有文件名
echo "*"          # 双引号屏蔽通配符展开 → 原样输出 *
echo '*'          # 单引号同样原样输出 *
```

把星号存进变量，观察**展开发生在哪一步**：

```
var3="*"
echo $var3        # 不加引号：变量替换后 * 再次被通配符展开 → 输出文件名列表
echo "$var3"      # 双引号：只做变量替换，结果 * 不再展开 → 输出 *
echo '$var3'      # 单引号：连变量替换都不做 → 原样输出 $var3
```

命令替换用反引号（或 `$()`）：

```
var4=`ls -l`      # 反引号内命令先执行，输出整体赋给 var4（含换行）
echo "$var4"      # 双引号保住换行，多行原样显示
echo $var4        # 不加引号，换行被折叠成空格，多行挤成一行

var5=ls
echo "`$var5`"    # 先变量替换成 ls，再作为命令执行，等价于 ls 的输出
```

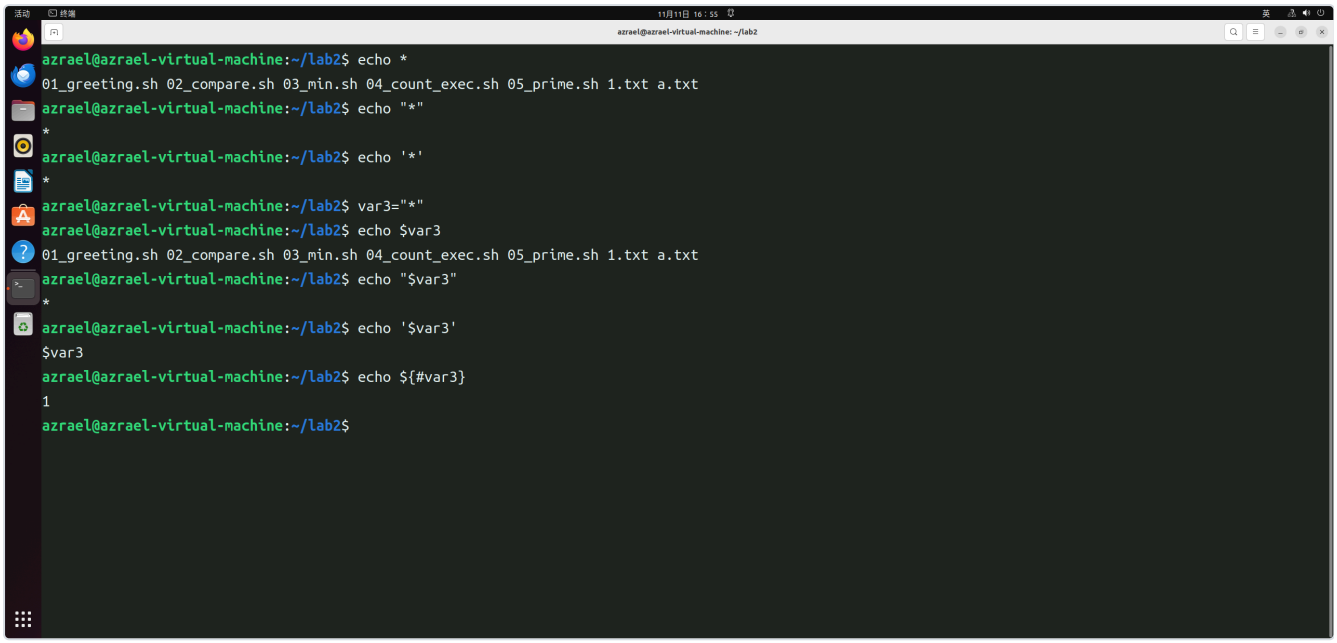


图 4-2 `echo *` 展开成文件名列表，而 `echo "*"`、`echo "$var3"` 输出 `*`；`echo $var3` 又展开，`echo '$var3'` 原样输出 `$var3`

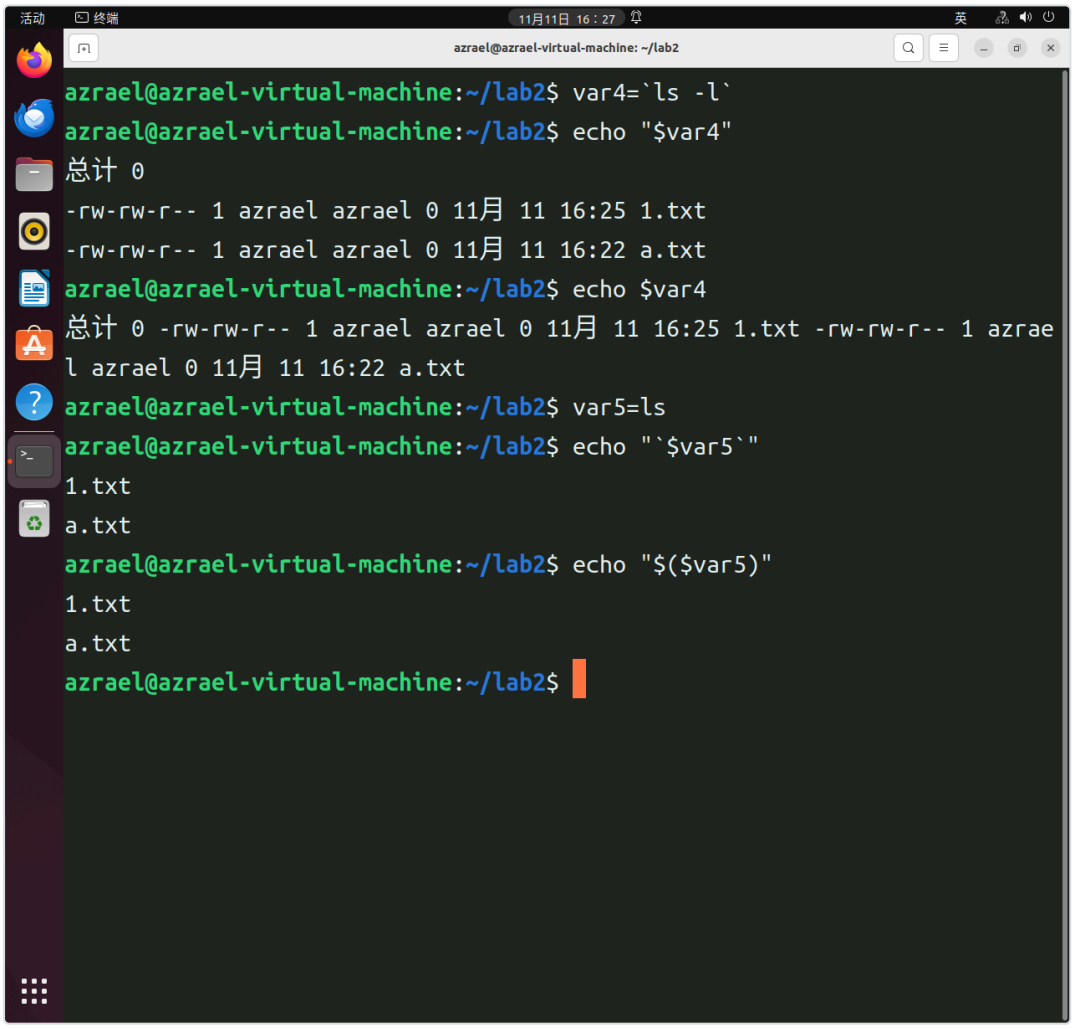


图 4-3 `var4` 用反引号取 `ls -l` 的输出后：`echo "$var4"` 保留换行，`echo $var4` 折叠成一行；反引号与 `$()` 两种命令替换写法结果一致

4.2.2 三种引号的作用一览

写法	变量替换	命令替换	通配符展开	空格与换行	典型用途
不加引号	做	做	展开	被字段拆分（IFS）	明确知道结果无空格时
`"双引号"`	做	做	**不展开**	**保留**	变量里可能含空格/换行（最常用）
`'单引号'`	**不做**	**不做**	**不展开**	保留	需要字面量、正则、`$` 符号本身

结论：展开顺序是「先命令替换/变量替换，再做通配符展开和字段拆分」，引号的作用就是在这个链条上逐层设卡：单引号全冻结，双引号只冻结通配符与字段拆分，不加引号则全部放行。因此脚本里凡是用到变量，**一律加双引号**是最稳妥的写法。表中「变量替换」指 `$var`，「命令替换」指反引号与 `$()`，「通配符展开」指 `*`、`?`、`[ ]`。

4.3 控制语句

4.3.1 判断系统时间的问候脚本 `01_greeting.sh`

需求：取系统当前小时，00–11 输出 `Good morning !!`，12–17 输出 `Good afternoon !!`，其余输出 `Good evening !!`。

```
#!/bin/bash
hour=`date +%H`
case $hour in
    0[0-9] | 1[01] )
        echo "Good morning !!"
        ;;
    1[2-7] )
        echo "Good afternoon !!"
        ;;
    * )
        echo "Good evening !!"
        ;;
esac
```

要点：

- `date +%H` 取两位小时（00–23），因为用了反引号，必须先执行命令再把结果赋给 `hour`；
- **赋值号两侧不能有空格**：若写成 `hour` 后面带空格再加 `=` 和空格，shell 会把 `hour` 当成命令名，报 `hour: command not found`；
- `case` 的分支用 `)` 结尾、以 `;;` 结束，模式之间用 `|` 表示「或」，`[0-9]` 是通配符字符范围，不是正则；
- 执行脚本必须写 `./01_greeting.sh`：当前目录 `.` 不在 `PATH` 里，直接敲 `01_greeting.sh` 会报「未找到命令」（见图 4-4）。
- 分支里也可以用 `[` 条件的 `if` 替代 `case`，但 `case` 更直观。

运行：

```
date +%H
./01_greeting.sh
```

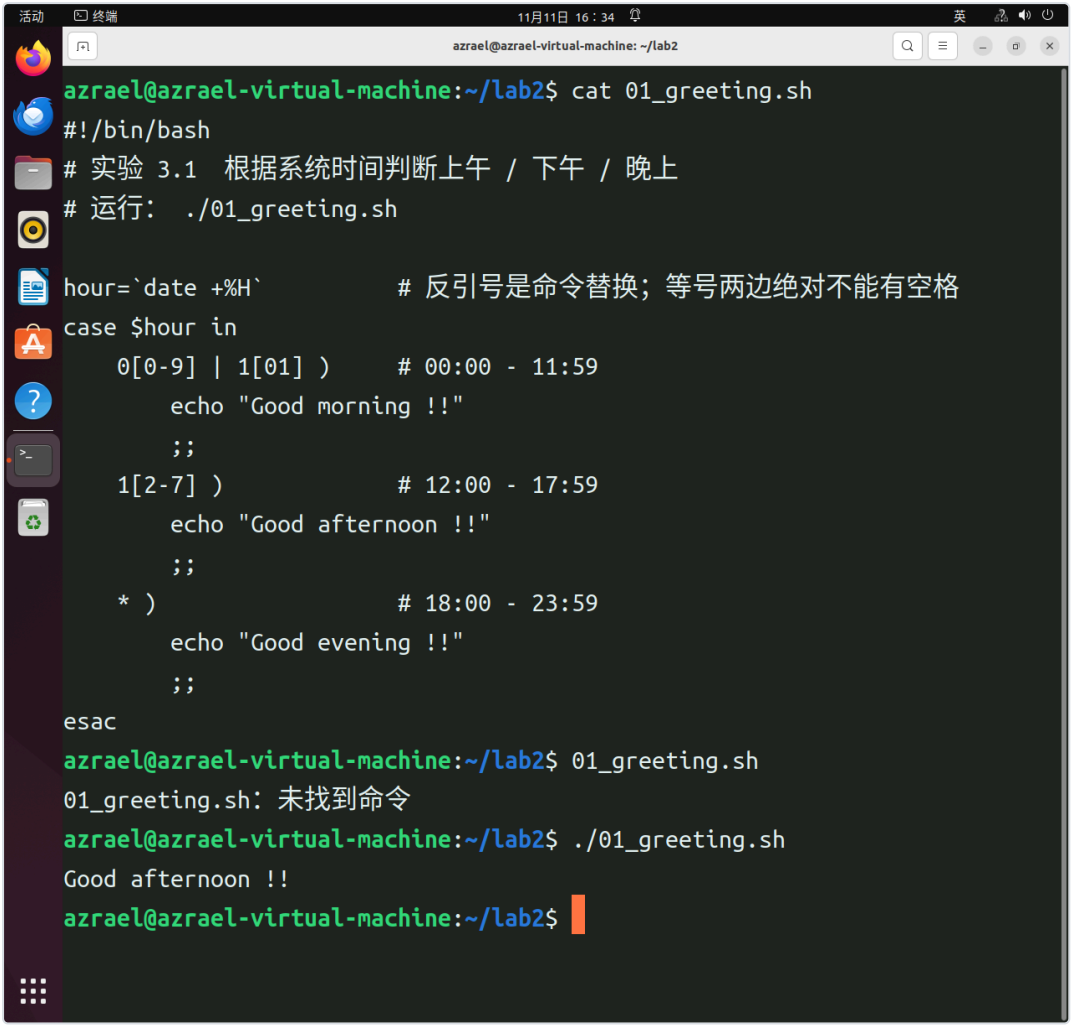


图 4-4　`cat 01_greeting.sh` 的脚本内容与 `./01_greeting.sh` 的输出（当时 16 点 → Good afternoon !!）；直接敲 `01_greeting.sh` 会报「未找到命令」

4.3.2 比较两个整数 `02_compare.sh`

```
#!/bin/bash
echo "Enter the first integer:"
read first
echo "Enter the second integer:"
read second

if [ "$first" -gt "$second" ]
then
    echo "$first is greater than $second"
elif [ "$first" -lt "$second" ]
then
    echo "$first is less than $second"
else
    echo "$first is equal to $second"
fi
```

要点：

- `read 变量` 从标准输入读一行赋值，提示语要单独 `echo` 或使用 `read -p`；
- `-gt` / `-lt` / `-eq` 用于**整数**比较，字符串比较要用 `>` `<` `=` 并注意转义；
- 变量必须加双引号，否则输入为空时 `[ -gt ]` 会报 `unary operator expected`；
- 讲义原稿里的 `$FIRST` 是大小写笔误，Shell 变量区分大小写，会展开为空串，必须统一成 `$first`。

运行三次，分别覆盖三种分支：

```
./02_compare.sh     # 114 / 514 → 114 is less than 514
./02_compare.sh     # 514 / 114 → 514 is greater than 114
./02_compare.sh     # 11 / 11   → 11 is equal to 11
```

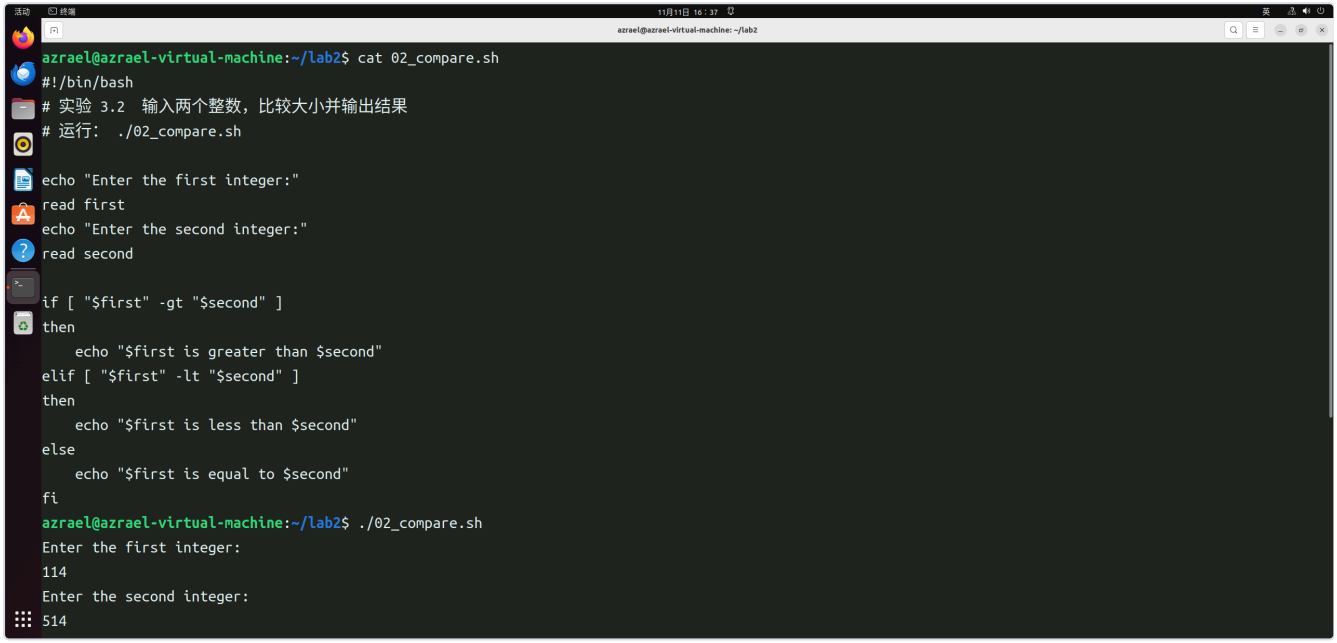


图 4-5　`02_compare.sh` 源码：`read` 两次输入 + `if/elif/else` 三路判断

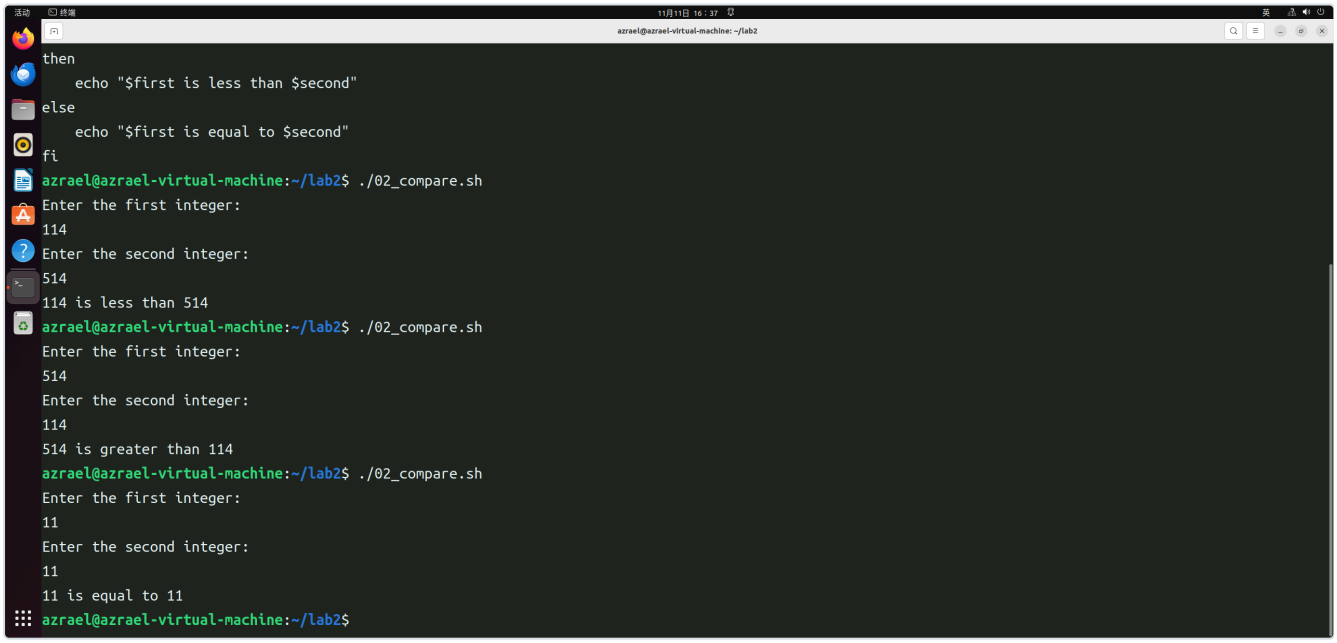


图 4-6　三次运行分别覆盖 less than / greater than / equal to 三个分支

4.3.3 从若干数中找最小值 `03_min.sh`

```
#!/bin/bash
smallest=10000
for i in 8 2 18 0 -3 87
do
    if test "$i" -lt "$smallest"
    then
        smallest=$i
    fi
done
echo $smallest
```

要点：

- 用「打擂台」思想：初始值取一个足够大的数（比所有待比较数都大），遍历时发现更小的就更新；
- `for i in 8 2 18 0 -3 87` 直接枚举列表，也可以用 `for i in $*` 或 `for i in $(cat file)`；
- `test 表达式` 与 `[ 表达式 ]` 完全等价，脚本里两种写法都可以；
- `-3` 能被正确识别为负数，因为 `-lt` 做的是数值比较而不是字符串比较（这里也说明必须用 `-lt` 而不能用 `<`）。

运行结果输出 `-3`：

```
./03_min.sh        # -3
```

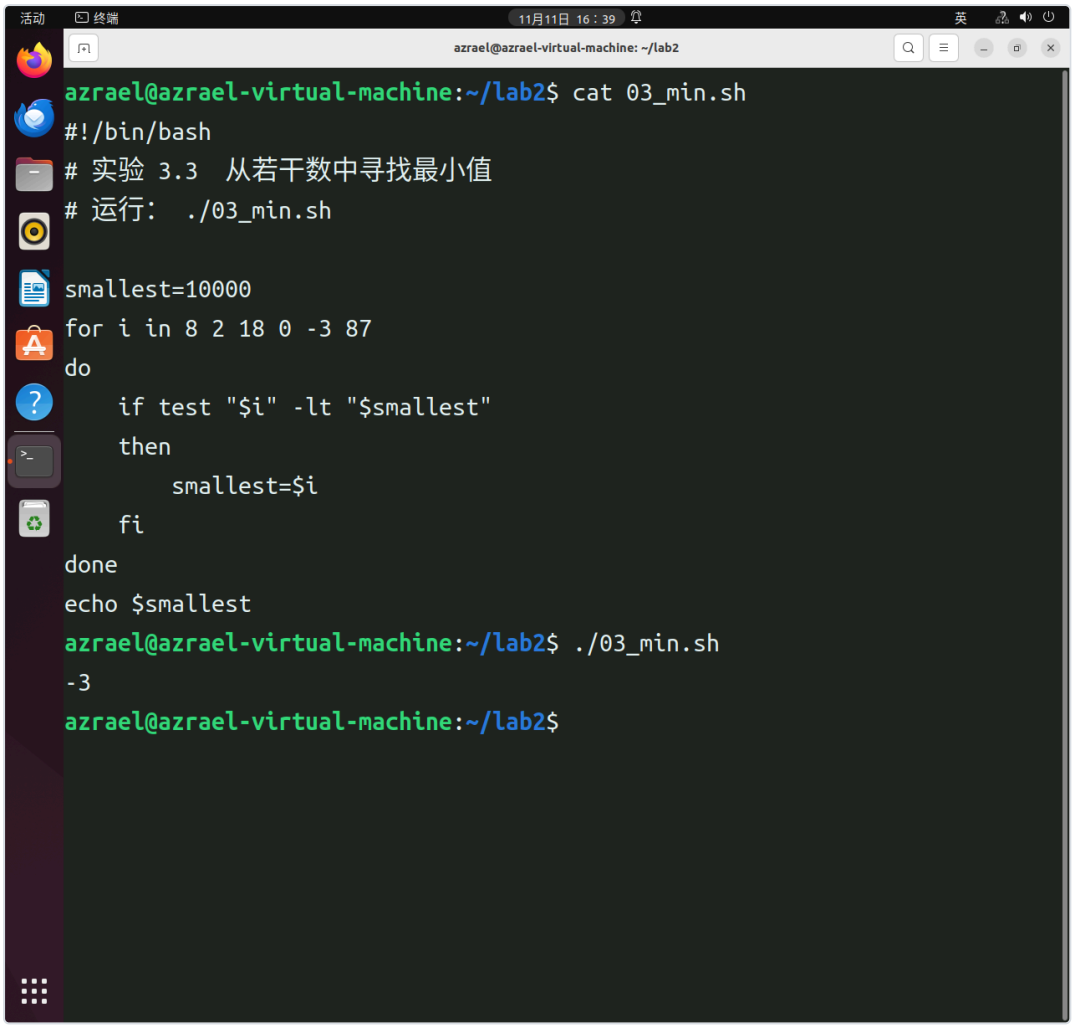


图 4-7　`03_min.sh` 输出 -3，即 8 2 18 0 -3 87 中的最小值

4.3.4 统计当前目录下可执行文件个数 `04_count_exec.sh`

```
#!/bin/bash
count=0
for i in *
do
    if [ -f "$i" ] && [ -x "$i" ]
    then
        count=`expr $count + 1`
    fi
done
echo "Total of $count files executable"
```

要点：

- `for i in *` 的 `*` 由 shell 展开成当前目录下的所有名字（等价于 `ls`），文件名中若含空格会带来隐患，更严谨的写法是 `for i in ./*`；
- `test -x` 判断文件是否可执行，但**目录天生带 x 权限**，如果只写 `test -x $i`，子目录也会被统计进去，因此加上 `-f` 先限定普通文件；
- `expr` 做算术运算时**运算符两侧必须有空格**：`expr $count + 1`；写成 `expr $count+1` 会把 `+1` 当成字符串拼起来；

- 现在更推荐 `count=$((count + 1))` 或 `let count++`，`expr` 是旧写法但很多教材仍在用。

运行（本目录下刚生成的 5 个 `.sh` 都带 x 权限，期望 5）：

```
cd ~/lab2
ls -l
./04_count_exec.sh     # Total of 5 files executable
```

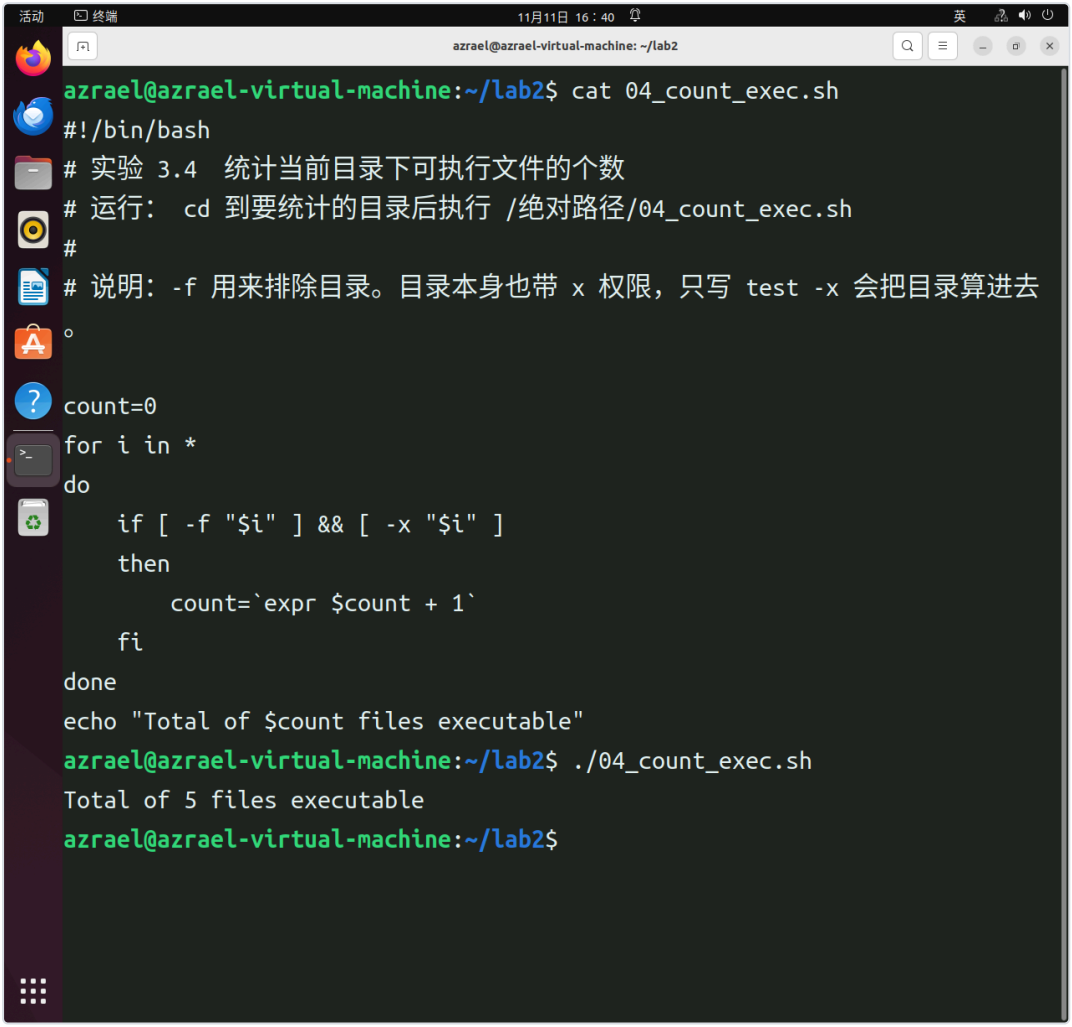


图 4-8　`ls -l` 中带 x 权限的 5 个脚本与脚本统计结果一致

4.4 函数：判断素数 `05_prime.sh`

```
#!/bin/bash
prime()
{
    flag=1
    j=2
    while [ $j -le `expr $1 / 2` ]
    do
        if [ `expr $1 % $j` -eq 0 ]
        then
            flag=0
            break
        fi
        j=`expr $j + 1`
    done

    if [ $flag -eq 1 ]
    then
        return 1
    else
        return 0
    fi
}

if [ $# -ne 1 ]
then
    echo "Usage: $0 <number>"
    exit 1
fi

prime $1
if [ $? -eq 1 ]
then
    echo "$1 is a prime!"
else
    echo "$1 is not a prime!"
fi
```

要点与说明：

1. **定义与调用**：`函数名()` 定义，`函数名 参数…` 调用；函数体里用 `$1`、`$2` 取**函数自己的**位置参数，与脚本的位置参数同名但作用域不同。
2. **返回值**：Shell 函数不能用 `return` 返回任意数值，只能返回 **0–255 的退出状态码**，用 `$?` 取到。本实验约定「1 表示是素数、0 表示不是素数」，所以调用后用 `[ $? -eq 1 ]` 判断。
   - 注意 `$?` 是**上一条命令**的状态码，中间不能再插入别的命令，否则会被覆盖。
   - 若确实要返回字符串或数值，应改用 `echo` 到标准输出 + 命令替换：`result=$(prime 7)`。
3. **算法**：试除法，只试到 `n/2`（甚至试到 `√n` 就够），发现能整除立即 `flag=0` 并 `break` 提前结束循环。
4. **边界**：讲义算法对 `1`、`0` 会误判为素数（`expr 1 / 2` 得 0，循环体一次都不执行，`flag` 保持 1）。若要严谨，可在函数开头加一行 `if [ "$1" -le 1 ]; then return 0; fi`，把 1 和 0 直接判为非素数。

5. 函数内的 `flag`、`j` 是**全局变量**，会污染外部同名变量；严谨写法是声明 `local j=2`。

运行验证：

```
./05_prime.sh 7       # 7 is a prime!
./05_prime.sh 99      # 99 is not a prime!
./05_prime.sh 114     # 114 is not a prime!
./05_prime.sh 1       # 讲义算法输出 1 is a prime!（已知缺陷，见上）
```

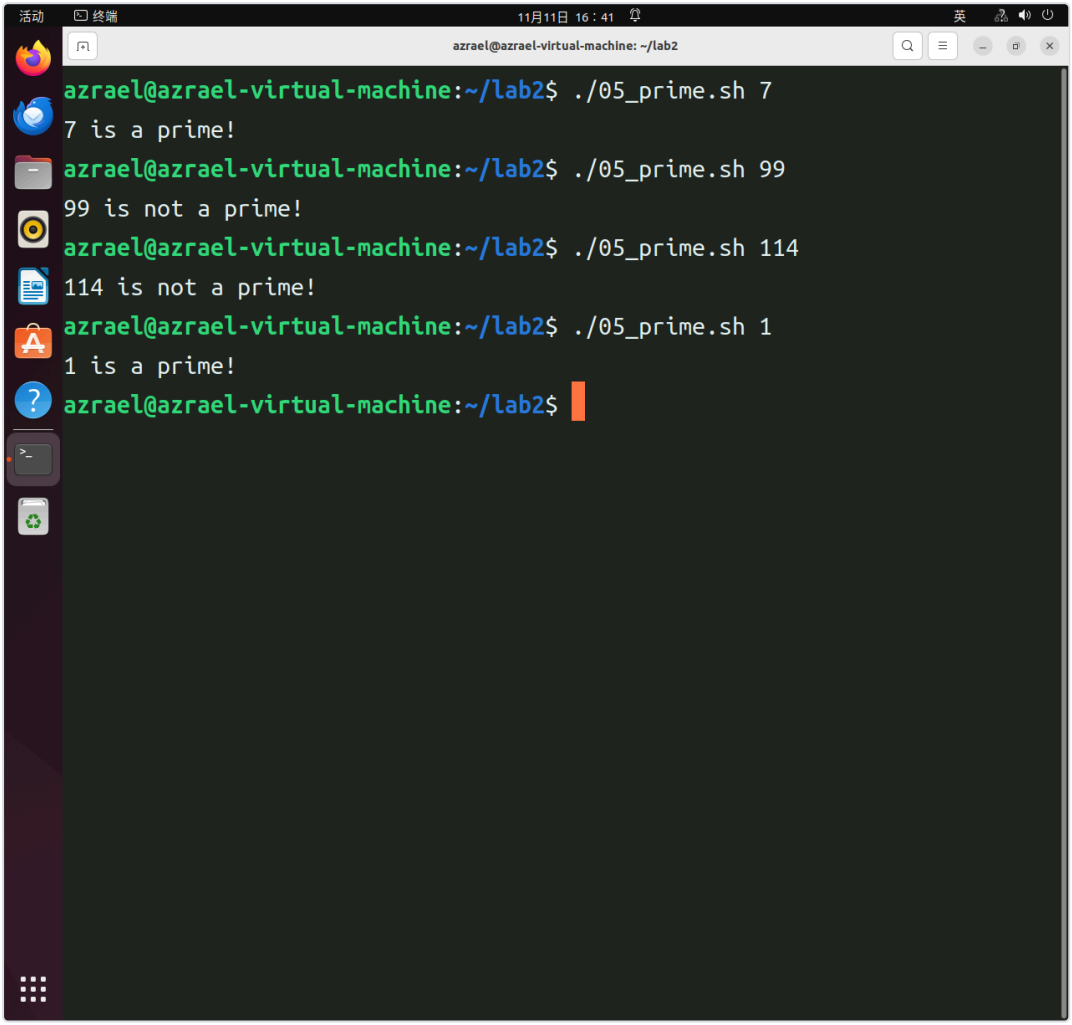


图 4-9　同一函数分别以 7、99、114、1 为参数调用的输出，验证返回值与 `$?` 判断；最后一条正是 1 的误判

4.5 讲义示例代码中的问题与修正

讲义中是打印稿，部分符号在排版时被拆开，直接照抄会报语法错误。实验中逐条验证后汇总如下：

序号	讲义原文	问题	修正后
1	`hour = `date +%H``	赋值号两侧有空格，shell 把 `hour` 当命令执行	`hour=`date +%H``
2	`t h e n` / `d o` / `e s a c` 等	关键字被逐字符拆开，不是合法关键字	`then` / `do` / `esac`
3	`; ;`	`case` 分支结束符中间不能有空格	`;;`
4	`echo "$FIRST ..."`	变量大小写不一致，`$FIRST` 为空	`$first`

序号	讲义原文	问题	修正后
5	`p r i m e ( )`	函数名与括号被拆开	`prime()`
6	`f l a g = 1`、`j = 2`	赋值号两侧有空格	`flag=1`、`j=2`
7	`j = ` e x p r $ j + 1 ` `	空格 + 逐字符拆分	`j=`expr $j + 1``
8	`// 注意……`	`//` 不是 bash 注释，会被当作路径	用 `#` 注释
9	`1[234567] )`	覆盖 12–17 点，18 点以后落到 `*`，逻辑本身可用但可读性差	`1[2-7] )`
10	`test -x $i` 统计可执行文件	目录也带 x 权限，被误计入	`[ -f "$i" ] && [ -x "$i" ]`
11	`expr $1 / 2` 未处理 1 和 0	1、0 被判为素数	函数开头加 `[ "$1" -le 1 ] && return 0`
12	`Good morining !!`	morning 拼写错误	`Good morning !!`

4.6 脚本清单与权限

五个脚本统一放在 `~/lab2`，用 `chmod +x` 赋执行权限后可以直接 `./脚本名` 运行：

```
cd ~/lab2
chmod +x *.sh
ls -l *.sh
```

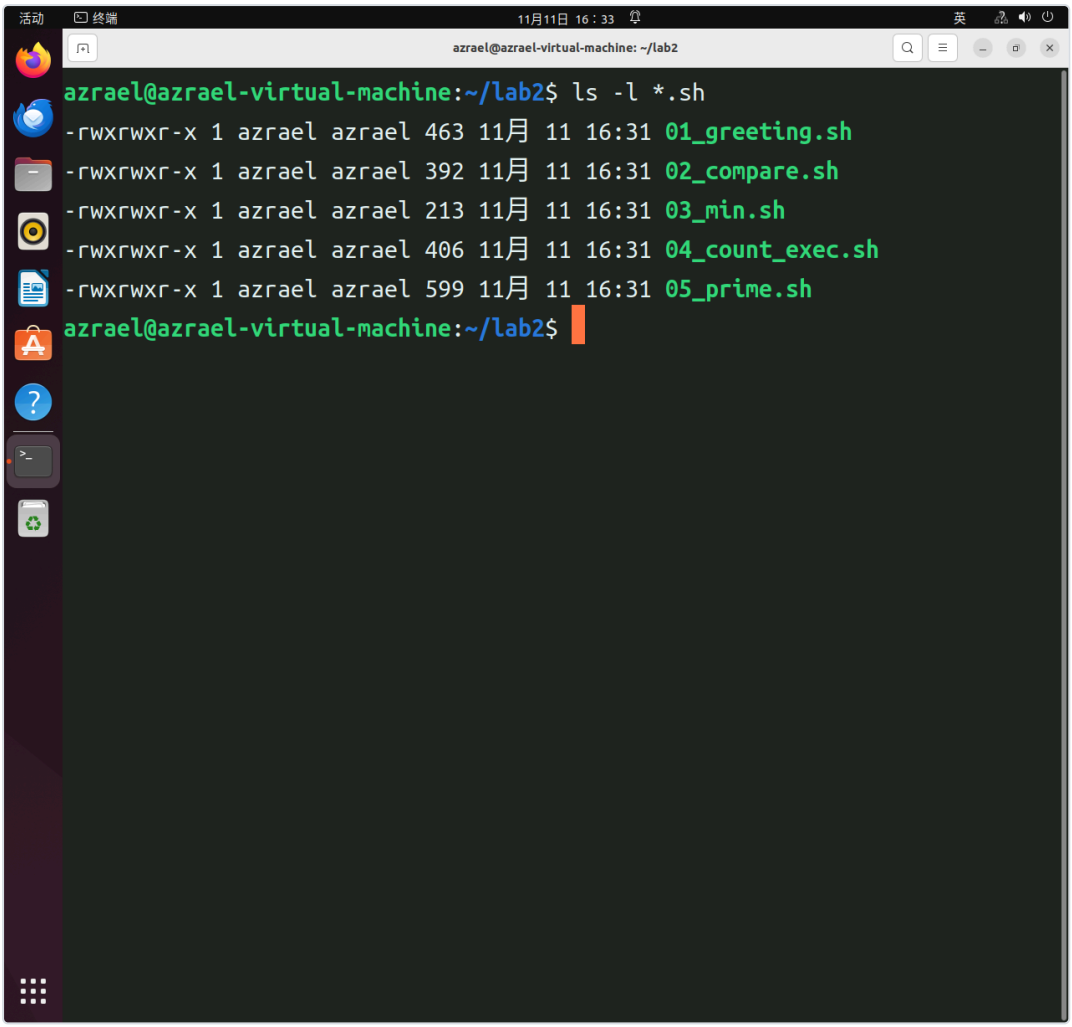


图 4-10　`ls -l *.sh`：五个脚本权限均为 `-rwxr-xr-x`

文件	对应讲义	功能
`01_greeting.sh`	第 3 部分第 1 题	按系统时间输出问候语（`case`）
`02_compare.sh`	第 3 部分第 2 题	比较两个整数大小（`if/elif/else` + `read`）
`03_min.sh`	第 3 部分第 3 题	一组数中求最小值（`for` + `test`）
`04_count_exec.sh`	第 3 部分第 4 题	统计当前目录可执行文件个数（`for` + `expr`）
`05_prime.sh`	第 3 部分第 5 题	用函数判断素数（`while` + 函数 + `$?`）

实验总结

1. **变量与作用域**：Shell 变量的可见性由「是否被 export」决定，而不是由声明位置决定。父进程导出的环境变量对子进程可见，子进程的修改不会回传，这一点和 C 语言里 fork 之后子进程有独立地址空间是同一个道理。
2. **引号是 Shell 的「展开开关」**：单引号全冻结、双引号冻结通配符与字段拆分、不加引号全部放行。写脚本时给变量统一加双引号，可以避免 90% 的空格、通配符导致的隐蔽 bug。
3. **控制语句与判断**：`if` 判断的是命令的**退出状态码**而不是布尔值，`test` / `[` 本质上只是一条返回 0/1 的命令，理解了这一点，`if grep -q x file; then` 这类写法就顺理成章。
4. **函数**：返回值只有 0–255 的状态码，真正的「返回数据」要靠标准输出或全局变量；`$?` 取的是上一条命令的状态，插入任何命令都会覆盖它。
5. **服务部署**：Redis 用 systemd 托管，`systemctl status` 一屏之内就能看到配置是否生效（监听地址、PID、状态文案），这是排查服务问题最快的入口；跨机访问要同时满足监听地址、保护模式、防火墙三项。
6. **工程习惯**：讲义代码不能照抄，遇到 `command not found`、`syntax error` 要能定位到「空格 / 大小写 / 逐字符拆分」这几类排版问题；脚本写完先 `bash -n` 做语法检查，再 `chmod +x` 执行。

附录　脚本源码

A.1 `01_greeting.sh`

```
#!/bin/bash
hour=`date +%H`
case $hour in
    0[0-9] | 1[01] )
        echo "Good morning !!"
        ;;
    1[2-7] )
        echo "Good afternoon !!"
        ;;
    * )
        echo "Good evening !!"
        ;;
esac
```

A.2 `02_compare.sh`

```
#!/bin/bash
echo "Enter the first integer:"
read first
echo "Enter the second integer:"
read second

if [ "$first" -gt "$second" ]
then
    echo "$first is greater than $second"
elif [ "$first" -lt "$second" ]
then
    echo "$first is less than $second"
else
    echo "$first is equal to $second"
fi
```

A.3 `03_min.sh`

```
#!/bin/bash
smallest=10000
for i in 8 2 18 0 -3 87
do
    if test "$i" -lt "$smallest"
    then
        smallest=$i
    fi
done
echo $smallest
```

A.4 04_count_exec.sh

```
#!/bin/bash
count=0
for i in *
do
    if [ -f "$i" ] && [ -x "$i" ]
    then
        count=`expr $count + 1`
    fi
done
echo "Total of $count files executable"
```

A.5 05_prime.sh

```
#!/bin/bash
prime()
{
    flag=1
    j=2
    while [ $j -le `expr $1 / 2` ]
    do
        if [ `expr $1 % $j` -eq 0 ]
        then
            flag=0
            break
        fi
        j=`expr $j + 1`
    done

    if [ $flag -eq 1 ]
    then
        return 1
    else
        return 0
    fi
}

if [ $# -ne 1 ]
then
    echo "Usage: $0 <number>"
    exit 1
fi

prime $1
if [ $? -eq 1 ]
then
    echo "$1 is a prime!"
else
    echo "$1 is not a prime!"
fi
```